

```
265
266 -- SECTION 4: ANALYTICAL QUERIES (DQL)
267
268 -- 1. Top 5 cities by student enrollments.
269
270 SELECT TOP 5
271 S.CITY,
272 COUNT(E.ENROLLMENTID)
273 AS TOTAL_ENROLLMENTS
274 FROM STUDENTS S
275 JOIN ENROLLMENTS E
276 ON S.STUDENTID = E.STUDENTID
277 GROUP BY S.CITY
278 ORDER BY TOTAL_ENROLLMENTS DESC
279
280 -- 2. Students enrolled in more than 2 courses.
281
282 SELECT STUDENTID,
283 COUNT(COURSEID) AS TOTALCOURSES
284 FROM ENROLLMENTS
```

78 %



99+



0



Results



Messages

	CITY	TOTAL_ENROLLMENTS
1	Hyderabad	141
2	Ahmedabad	135
3	Goa	131
4	Delhi	118
5	Jaipur	117

SQLProject1....ithika (65))*

```
276 ON S.STUDENTID = E.STUDENTID
277 GROUP BY S.CITY
278 ORDER BY TOTAL_ENROLLMENTS DESC
279
280 -- 2. Students enrolled in more than 2 courses.
281
282 SELECT STUDENTID,
283 COUNT (COURSEID) AS TOTALCOURSES
284 FROM ENROLLMENTS
285 GROUP BY STUDENTID
286 HAVING COUNT(COURSEID) > 2
287
288 -- 3. Students with no payment records.
289 SELECT * FROM ENROLLMENTS
```

78 %

99+ 0



Results Messages

	STUDENTID	TOTALCOURSES
1	1804	3
2	1211	3
3	1397	3
4	1188	3
5	1420	3
6	1117	4
7	1781	4
8	1658	3
9	1466	3
10	1251	3
11	1263	3
12	1449	4
13	1303	4
14	1838	3
15	1492	5


```
288      -- 3. Students with no payment records.
289      SELECT * FROM ENROLLMENTS
290      SELECT * FROM PAYMENTS
291
292      SELECT DISTINCT STUDENTID
293      FROM ENROLLMENTS
294      WHERE ENROLLMENTID NOT IN
295      (
296      SELECT ENROLLMENTID FROM PAYMENTS
297      )
298
299
```

STUDENTID	
1	1001

```
300
301 -- 4. Highest revenue generating month.
302
303 SELECT TOP 1
304     MONTH(PaymentDate) AS RevenueMonth,
305     SUM(AmountPaid) AS TotalRevenue
306 FROM PAYMENTS
307 GROUP BY MONTH(PaymentDate)
308 ORDER BY TotalRevenue DESC
309
310 --5. Top 3 most popular courses.
311
```

	RevenueMonth	TotalRevenue
1	3	4465133.00


```
300
301 -- 4. Highest revenue generating month.
302
303 SELECT TOP 1
304     MONTH(PaymentDate) AS RevenueMonth,
305     SUM(AmountPaid) AS TotalRevenue
306 FROM PAYMENTS
307 GROUP BY MONTH(PaymentDate)
308 ORDER BY TotalRevenue DESC
309
310 --5. Top 3 most popular courses.
311
```

	RevenueMonth	TotalRevenue
1	3	4465133.00

```
--5. Top 3 most popular courses.
```

```
SELECT TOP 3  
COURSENAME AS POPULARCOURSES, COUNT(E.STUDENTID) AS TOTALSTUDENTS  
FROM COURSES AS C  
JOIN ENROLLMENTS AS E  
ON C.COURSEID = E.COURSEID  
GROUP BY COURSENAME  
ORDER BY TOTALSTUDENTS DESC
```

86 %



99+



0



Ln: 320, Ch: 1

Results Messages

	POPULARCOURSES	TOTALSTUDENTS
1	Cloud Foundations	129
2	Generative AI	129
3	Python Full Stack	128

SQLProject1....ithika (65))*

--6. Enrollments with discount above average discount.

```
SELECT AVG(DISCOUNT) AS AVGDISCOUNT FROM ENROLLMENTS
```

```
SELECT ENROLLMENTID, DISCOUNT  
FROM ENROLLMENTS  
WHERE DISCOUNT > 15
```

--7. States with more than 10 paid enrollments.

86 %

99+ 0

Ln: 331, Ch: 20 (68 ch)

Results Messages

	ENROLLMENTID	DISCOUNT
1	2001	25.00
2	2003	25.00
3	2005	20.00
4	2006	30.00
5	2008	25.00
6	2009	25.00
7	2011	20.00
8	2012	25.00
9	2013	30.00
10	2016	25.00
11	2018	20.00
12	2020	25.00
13	2022	25.00
14	2023	20.00
15	2024	25.00

Query executed successfully.

localhost\SQLEXPRESS (17.0 ... RITHIKA\Rithika (65) EdTechLearnPro 00:00


```
--7. States with more than 10 paid enrollments.
```

```
SELECT S.STATE, COUNT(E.ENROLLMENTID)
AS TOTAL_PAID_ENROLLMENTS
FROM STUDENTS S
JOIN ENROLLMENTS E
ON S.STUDENTID = E.STUDENTID
WHERE E.PAYMENTSTATUS = 'PAID'
GROUP BY S.STATE
HAVING COUNT(E.ENROLLMENTID) > 10
```

86 %



99+



0



Ln: 344, Ch: 1 (21

Results Messages

	STATE	TOTAL_PAID_ENROLLMENTS
1	Delhi	34
2	Goa	47
3	Gujarat	52
4	Karnataka	33
5	Maharashtra	80
6	Rajasthan	30
7	Tamil Nadu	32
8	Telangana	52
9	West Bengal	41

SQLProject1....ithika (65))*

345

346

347

348

349

350

351

352

353

354

355

356

357

358

359

360

-- PART 5 - JOINS (ALL TYPES)

--Write JOIN queries that combine:

--Students + Enrollments + Courses

-- Display student enrollment details along with the courses they enrolled in and their enrollment date

SELECT

S.FULLNAME,

C.COURSENAME,

E.ENROLLMENTID,

E.ENROLLMENTDATE

FROM STUDENTS S

JOIN ENROLLMENTS E

ON S.STUDENTID = E.STUDENTID

JOIN COURSES C

86 %

99+

0

Ln: 362, Ch: 1 (183 chars, 10 lines)

Results

Messages

	FULLNAME	COURSENAME	ENROLLMENTID	ENROLLMENTDATE
1	Student_318	Machine Learning	2001	2025-03-22
2	Student_111	Generative AI	2002	2025-03-29
3	Student_251	Excel Power User	2003	2025-07-13
4	Student_383	SQL Mastery	2004	2025-04-14
5	Student_7	Excel Power User	2005	2025-09-28
6	Student_459	Cloud Foundations	2006	2025-04-06
7	Student_84	SQL Mastery	2007	2025-07-17
8	Student_208	Cloud Foundations	2008	2025-01-01
9	Student_900	Power BI	2009	2025-06-16
10	Student_390	Generative AI	2010	2025-05-06
11	Student_314	Power BI	2011	2025-11-09

Query executed successfully.

localhost\SQLEXPRESS (17.0 ...

RITHIKA\Rithika (65)

EdTechLearnPro

00:00:00

Row: 1, Col:


```
363 -- DISPLAY ALL STUDENTS ALONG WITH THEIR ENROLLED COURSES, INCLUDING STUDENTS WHO HAVE NOT ENROLLED YET
364
365 SELECT
366 S.STUDENTID,
367 E.COURSEID,
368 E.ENROLLMENTID
369 FROM STUDENTS AS S
370 LEFT JOIN ENROLLMENTS AS E
371 ON S.STUDENTID = E.STUDENTID
372 LEFT JOIN COURSES AS C
373 ON E.COURSEID = C.COURSEID
374 |
375 --Courses + TrainerCourseMapping + Trainers
376 --Trainer performance with number of assigned courses
377
378 SELECT * FROM COURSES
```

	STUDENTID	COURSEID	ENROLLMENTID
1	1318	507	2001
2	1111	508	2002
3	1251	510	2003
4	1383	506	2004
5	1007	510	2005
6	1459	509	2006
7	1084	506	2007
8	1208	509	2008
9	1900	504	2009
10	1390	508	2010
11	1314	504	2011

SQLProject1....ithika (65))*

375 --Courses + TrainerCourseMapping + Trainers
376 --Trainer performance with number of assigned courses
377
378 SELECT * FROM COURSES
379 SELECT * FROM TRAINER_COURSE_MAPPING
380 SELECT * FROM TRAINERS
381
382
383 SELECT
384 T.TRAINERNAME,
385 COUNT(M.COURSEID) AS TOTAL_ASSIGNED_COURSES
386 FROM COURSES C
387 RIGHT JOIN TRAINER_COURSE_MAPPING M
388 ON C.COURSEID = M.COURSEID
389 RIGHT JOIN TRAINERS T
390 ON M.TRAINERID = T.TRAINERID

86 % 99+ 0 Ln: 391, Ch: 24 (234 chars, 9 lines)

Results Messages

	TRAINERNAME	TOTAL_ASSIGNED_COURSES
1	Trainer_1	5
2	Trainer_10	1
3	Trainer_2	4
4	Trainer_3	1
5	Trainer_4	1
6	Trainer_5	1
7	Trainer_6	3
8	Trainer_7	0
9	Trainer_8	4
10	Trainer_9	4

Query executed successfully. localhost\SQL EXPRESS (17.0 ... RITHIKA\Rithika (65) EdTechLearnPro 00:00:00 Row:


```
-- Enrollments + Payments
```

```
SELECT E.COURSEID,  
COUNT(E.ENROLLMENTID) AS  
TOTAL_ENROLLMENTS,  
SUM(P.AMOUNTPAID) AS TOTAL_REVENUE  
FROM ENROLLMENTS E  
INNER JOIN PAYMENTS P  
ON E.ENROLLMENTID = P.ENROLLMENTID  
GROUP BY E.COURSEID  
ORDER BY TOTAL_REVENUE DESC
```

```
-- REVENUE PER COURSE
```

```
SELECT
```

86 %

99+

0

↑ ↓

Results Messages

	COURSEID	TOTAL_ENROLLMENTS	TOTAL_REVENUE
1	501	123	4747754.00
2	504	121	4469093.00
3	503	127	4458602.00
4	508	129	4334813.00
5	506	123	4261140.00
6	509	129	4244354.00
7	505	128	4143598.00
8	507	107	3880273.00
9	502	111	3741912.00
10	510	103	3476370.00


```
405
406      -- REVENUE PER COURSE
407
408      SELECT
409      E.COURSEID,
410      SUM(P.AMOUNTPAID) AS TOTAL_REVENUE
411      FROM ENROLLMENTS AS E
412      LEFT JOIN PAYMENTS AS P
413      ON E.ENROLLMENTID = P.ENROLLMENTID
414      GROUP BY E.COURSEID
415
416      -- DISPLAY DISCOUNT FOR EACH COURSE WITH PAYMENT AMOUNT
417
418      SELECT
419      E.COURSEID, SUM(E.DISCOUNT) AS TOTAL_DISCOUNT, SUM(P.AMOUNTPAID) AS TOTAL_AMOUNT
420      FROM ENROLLMENTS AS E
```

86 %

99+ 0



Ln: 414, Ch: 21

Results Messages

	COURSEID	TOTAL_REVENUE
1	507	3880273.00
2	503	4458602.00
3	510	3476370.00
4	502	3741912.00
5	505	4143598.00
6	509	4244354.00
7	508	4334813.00
8	504	4469093.00
9	501	4747754.00
10	506	4261140.00

✓ Query executed successfully.

localhost\SQLEXPRESS (17.0 ... RITHIKA\Rithika (65) EdTechLearnP


```
414 GROUP BY E.COURSEID
415
416 -- DISPLAY DISCOUNT FOR EACH COURSE WITH PAYMENT AMOUNT
417
418 SELECT
419 E.COURSEID, SUM(E.DISCOUNT) AS TOTAL_DISCOUNT, SUM(P.AMOUNTPAID) AS TOTAL_AMOUNT
420 FROM ENROLLMENTS AS E
421 LEFT JOIN PAYMENTS AS P
422 ON E.ENROLLMENTID = P.ENROLLMENTID
423 GROUP BY E.COURSEID
424 ORDER BY E.COURSEID ASC
425
426 -- Marketing + Enrollments (month based)
427 SELECT * FROM MARKETING
428 SELECT * FROM ENROLLMENTS
429
```

86 %



99+



0



Ln: 424, Ch: 25

Results Messages

	COURSEID	TOTAL_DISCOUNT	TOTAL_AMOUNT
1	501	1850.00	4747754.00
2	502	1665.00	3741912.00
3	503	1900.00	4458602.00
4	504	1975.00	4469093.00
5	505	1965.00	4143598.00
6	506	2000.00	4261140.00
7	507	1685.00	3880273.00
8	508	1945.00	4334813.00
9	509	2115.00	4244354.00
10	510	1640.00	3476370.00

Query executed successfully.



localhost\SQLEXPRESS (17.0 ...

RITHIKA\Rithika (65)

EdTechLearnPro

SQLProject1....ithika (65))*

426

-- Marketing + Enrollments (month based)

427

SELECT * FROM MARKETING

428

SELECT * FROM ENROLLMENTS

429

SELECT

430

M.MONTHYEAR,

431

M.CHANNEL,

432

M.LEADSGENERATED,

433

COUNT(E.ENROLLMENTID) AS TOTAL_ENROLLMENTS,

434

M.SPEND,

435

ROUND(

436

COUNT(E.ENROLLMENTID) * 100.0 /

437

NULLIF(M.LEADSGENERATED,0),2

438

) AS CONVERSION_RATE

439

FROM MARKETING M

440

LEFT JOIN ENROLLMENTS E

441

ON FORMAT(E.ENROLLMENTDATE, 'yyyy-MM') = M.MONTHYEAR

86 %

7

0

↑

↓

Results

Messages

	MONTHYEAR	CHANNEL	LEADSGENERATED	TOTAL_ENROLLMENTS	SPEND	CONVERSION_RATE
1	2025-01	Email	1051	84	168724.00	7.990000000000
2	2025-01	Google Ads	327	84	76853.00	25.690000000000
3	2025-01	Influencers	711	84	287777.00	11.810000000000
4	2025-01	LinkedIn	1026	84	231565.00	8.190000000000
5	2025-01	Meta Ads	1075	84	123719.00	7.810000000000
6	2025-01	YouTube	357	84	201733.00	23.530000000000
7	2025-02	Email	514	93	210637.00	18.090000000000
8	2025-02	Google Ads	244	93	217567.00	38.110000000000
9	2025-02	Influencers	376	93	244393.00	24.730000000000
10	2025-02	LinkedIn	642	93	211887.00	14.490000000000
11	2025-02	Meta Ads	1331	93	63703.00	6.990000000000

Query executed successfully.

localhost\SQLEXPRESS (17.0 ...

RITHIKA\Rithika (65)

EdTechLe

SQLProject1....ithika (65))*

450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465

--MONTHWISE REVENUE
--Display channel-wise marketing performance with enrollments

SELECT M.MONTHYEAR, COUNT(E.ENROLLMENTID) AS TOTAL_ENROLLMENTS, M.CHANNEL, M.LEADSGENERATED, M.SPEND
FROM MARKETING AS M
LEFT JOIN ENROLLMENTS AS E
ON FORMAT(E.ENROLLMENTDATE, 'yyyy-MM') = M.MONTHYEAR
GROUP BY M.MONTHYEAR, M.CHANNEL, M.LEADSGENERATED, M.SPEND
ORDER BY M.MONTHYEAR

-- City wise + course wise revenue summary

86 %

7 0

Ln: 435, Ch: 11 | SPC | CF

Results

Messages

	MONTHYEAR	CHANNEL	LEADSGENERATED	TOTAL_ENROLLMENTS	SPEND	CONVERSION_RATE
1	2025-01	Email	1051	84	168724.00	7.990000000000
2	2025-01	Google Ads	327	84	76853.00	25.690000000000
3	2025-01	Influencers	711	84	287777.00	11.810000000000
4	2025-01	LinkedIn	1026	84	231565.00	8.190000000000
5	2025-01	Meta Ads	1075	84	123719.00	7.810000000000
6	2025-01	YouTube	357	84	201733.00	23.530000000000
7	2025-02	Email	514	93	210637.00	18.090000000000
8	2025-02	Google Ads	244	93	217567.00	38.110000000000
9	2025-02	Influencers	376	93	244393.00	24.730000000000
10	2025-02	LinkedIn	642	93	211887.00	14.490000000000
11	2025-02	Meta Ads	1331	93	63703.00	6.990000000000

Query executed successfully.

localhost\SQLEXPRESS (17.0 ... | RITHIKA\Rithika (65) | EdTechLearnPro | 00:00:11 | Row: 1, Col: 1

Select Repository


```
462
463
464      -- City wise + course wise revenue summary
465
466      SELECT * FROM STUDENTS
467      SELECT * FROM COURSES
468      SELECT * FROM ENROLLMENTS
469      SELECT * FROM PAYMENTS
470
471      SELECT S.CITY, C.COURSENAME, SUM(P.AMOUNTPAID) AS REVENUE
472      FROM STUDENTS AS S
473      JOIN ENROLLMENTS AS E
474      ON S.STUDENTID = E.STUDENTID
475      JOIN COURSES AS C
476      ON E.COURSEID = C.COURSEID
477      JOIN PAYMENTS AS P
```

86 %



7



0



Results Messages

	MONTHYEAR	CHANNEL	LEADSGENERATED	TOTAL_ENROLLMENTS	SPEND	CONVERSION_RATE
1	2025-01	Email	1051	84	168724.00	7.990000000000
2	2025-01	Google Ads	327	84	76853.00	25.690000000000
3	2025-01	Influencers	711	84	287777.00	11.810000000000
4	2025-01	LinkedIn	1026	84	231565.00	8.190000000000
5	2025-01	Meta Ads	1075	84	123719.00	7.810000000000
6	2025-01	YouTube	357	84	201733.00	23.530000000000
7	2025-02	Email	514	93	210637.00	18.090000000000
8	2025-02	Google Ads	244	93	217567.00	38.110000000000
9	2025-02	Influencers	376	93	244393.00	24.730000000000
10	2025-02	LinkedIn	642	93	211887.00	14.490000000000
11	2025-02	Meta Ads	1331	93	63703.00	6.990000000000

Query executed successfully.



localhost\SQLEXPRESS (17.0 ...

RITHIKA\Rithika (65)

EdTechLearnPro


```
464      -- City wise + course wise revenue summary
465
466      SELECT * FROM STUDENTS
467      SELECT * FROM COURSES
468      SELECT * FROM ENROLLMENTS
469      SELECT * FROM PAYMENTS
470
471      SELECT S.CITY, C.COURSENAME, SUM(P.AMOUNTPAID) AS REVENUE
472      FROM STUDENTS AS S
473      JOIN ENROLLMENTS AS E
474      ON S.STUDENTID = E.STUDENTID
475      JOIN COURSES AS C
476      ON E.COURSEID = C.COURSEID
477      JOIN PAYMENTS AS P
478      ON E.ENROLLMENTID = P.ENROLLMENTID
479      GROUP BY S.CITY, C.COURSENAME
```

	CITY	COURSENAME	REVENUE
1	Ahmedabad	Python Full Stack	702447.00
2	Ahmedabad	Power BI	642560.00
3	Ahmedabad	Excel Power User	567968.00
4	Ahmedabad	Data Science	485895.00
5	Ahmedabad	Machine Learning	482512.00
6	Ahmedabad	Business Analytics	473401.00
7	Ahmedabad	Data Analytics	462347.00
8	Ahmedabad	Generative AI	433527.00
9	Ahmedabad	SQL Mastery	395417.00
10	Ahmedabad	Cloud Foundations	391358.00
11	Bangalore	Data Analytics	455578.00


```
482
483      --Student Name + Course Name + Trainer Name
484
485
486      SELECT S.FULLNAME, C.COURSENAME, T.TRAINERNAME
487      FROM STUDENTS S
488      JOIN ENROLLMENTS E ON S.STUDENTID = E.STUDENTID
489      JOIN COURSES C ON E.COURSEID = C.COURSEID
490      JOIN TRAINER_COURSE_MAPPING M ON C.COURSEID = M.COURSEID
491      JOIN TRAINERS T ON M.TRAINERID = T.TRAINERID
492
493
494      --PART 6 - GROUP BY + HAVING + AGGREGATION
495      --- Monthly revenue
496
497      SELECT * FROM PAYMENTS
```

86 %



7



0



Results Messages

	FULLNAME	COURSENAME	TRAINERNAME
1	Student_318	Machine Learning	Trainer_1
2	Student_318	Machine Learning	Trainer_2
3	Student_318	Machine Learning	Trainer_3
4	Student_318	Machine Learning	Trainer_9
5	Student_111	Generative AI	Trainer_1
6	Student_111	Generative AI	Trainer_2
7	Student_111	Generative AI	Trainer_9
8	Student_251	Excel Power User	Trainer_1
9	Student_251	Excel Power User	Trainer_6
10	Student_251	Excel Power User	Trainer_8
11	Student_383	SQL Mastery	Trainer_5

Query executed successfully.



localhost\SQLEXPRESS (17.0 ... RITHIKA

SQLProject1....ithika (65))*

```
494 --PART 6 - GROUP BY + HAVING + AGGREGATION
495 --- Monthly revenue
496
497 SELECT * FROM PAYMENTS
498
499 SELECT SUM(AMOUNTPAID) AS TOTAL_REVENUE, MONTH(PAYMENTDATE) AS MONTH
500 FROM PAYMENTS
501 GROUP BY MONTH(PAYMENTDATE)
502 HAVING SUM(AMOUNTPAID) > 0
503 ORDER BY MONTH(PAYMENTDATE)
504
505 --- Enrollment counts per course
506 SELECT * FROM ENROLLMENTS
507
508 SELECT COUNT(ENROLLMENTID) AS ENROLLMENTS, COURSEID
509 FROM ENROLLMENTS
```

86 %

7 0

↑ ↓

Results Messages

	TOTAL_REVENUE	MONTH
1	4337543.00	1
2	3716072.00	2
3	4465133.00	3
4	3855806.00	4
5	2952099.00	5
6	3192431.00	6
7	3047863.00	7
8	3311610.00	8
9	3001606.00	9
10	3301073.00	10
11	3211283.00	11

Query executed successfully.

localhost\SQLEXPRESS (17.0 ... RITHIKA\Rithika (65)

SQLProject1....ithika (65))*

```
500 FROM PAYMENTS
501 GROUP BY MONTH(PAYMENTDATE)
502 HAVING SUM(AMOUNTPAID) > 0
503 ORDER BY MONTH(PAYMENTDATE)
504
505 --- Enrollment counts per course
506 SELECT * FROM ENROLLMENTS
507
508 SELECT COUNT(ENROLLMENTID) AS ENROLLMENTS, COURSEID
509 FROM ENROLLMENTS
510 GROUP BY COURSEID
511 HAVING COUNT(ENROLLMENTID) > 0
512 ORDER BY COURSEID ASC
513
514 -- Average fee per category
515
```

86 %

7

0

Ln: 51

Results Messages

	ENROLLMENTS	COURSEID
1	124	501
2	111	502
3	127	503
4	121	504
5	128	505
6	123	506
7	107	507
8	129	508
9	129	509
10	103	510

Query executed successfully.

localhost\SQLEXPRESS (17.0 ... RITHIKA\Rithika (65) EdTe


```
512 ORDER BY COURSEID ASC
513
514 -- Average fee per category
515
516 SELECT * FROM COURSES
517
518 SELECT AVG(COURSEFEE) AS AVGCOURSE_FEE, CATEGORY
519 FROM COURSES
520 GROUP BY CATEGORY
521 HAVING AVG(COURSEFEE) > 1000
522 ORDER BY CATEGORY |
523
524 -- Trainer satisfaction averages
525 SELECT * FROM TRAINERS
526
527 SELECT CAST(ROUND(AVG(SATISFACTIONSORE),2) AS DECIMAL(10,2)) AS AVGSATISFACTIONSORE, EXPERTISE, TRAINERNAME
```

	AVGCOURSE_FEE	CATEGORY
1	70000	AI
2	47500	Analytics
3	30000	BI
4	40000	Cloud
5	25000	Databases
6	20000	Productivity
7	55000	Programming


```
521     HAVING AVG(COURSEFEE) > 1000
522     ORDER BY CATEGORY
523
524     -- Trainer satisfaction averages
525     SELECT * FROM TRAINERS
526
527     SELECT CAST(ROUND(AVG(SATISFACTIONSORE),2) AS DECIMAL(10,2)) AS AVGSATISFACTIONSORE, EXPERTISE, TRAINERNAME
528     FROM TRAINERS
529     GROUP BY TRAINERNAME, EXPERTISE
530     HAVING ROUND(AVG(SATISFACTIONSORE),2) > 1
531     ORDER BY TRAINERNAME ASC
532
533     --State wise registration patterns
534
535     SELECT * FROM STUDENTS
536
```

86 %



7



0



Ln: 532, Ch: 1 (232 chars, 6 lin

Results Messages

	AVGSATISFACTIONSORE	EXPERTISE	TRAINERNAME
1	4.19	Power BI	Trainer_1
2	3.99	Cloud	Trainer_10
3	3.50	Python	Trainer_2
4	4.40	AI	Trainer_3
5	4.41	Power BI	Trainer_4
6	3.87	AI	Trainer_5
7	4.34	Power BI	Trainer_6
8	3.84	SQL	Trainer_7
9	3.74	Python	Trainer_8
10	4.48	Power BI	Trainer_9

Query executed successfully.



localhost\SQLEXPRESS (17.0 ...

RITHIKA\Rithika (65)

EdTechLearnPro

00:00:00 R




```
533 --State wise registration patterns
534
535 SELECT * FROM STUDENTS
536
537 SELECT STATE, COUNT(STUDENTID) AS TOTALREGISTRATIONS
538 FROM STUDENTS
539 GROUP BY STATE
540 ORDER BY TOTALREGISTRATIONS DESC
541
542 --PART 7 - SUBQUERIES + CTEs
543
544 -- Courses with above average enrollments
545
546 SELECT COURSEID, COUNT(ENROLLMENTID) AS TOTAL_ENROLLMENTS
547 FROM ENROLLMENTS
548 GROUP BY COURSEID
```

86 %



7



0



Ln: 537, Ch: 1 (117 chars, 4 lines) SPC CRLF UTF-8

Results Messages

	STATE	TOTALREGISTRATIONS
1	Maharashtra	166
2	Telangana	100
3	Gujarat	99
4	Karnataka	94
5	Tamil Nadu	94
6	Delhi	92
7	Goa	90
8	Rajasthan	84
9	West Bengal	82


```
540 ORDER BY TOTALREGISTRATIONS DESC
541
542 --PART 7 - SUBQUERIES + CTEs
543
544 -- Courses with above average enrollments
545
546 SELECT COURSEID, COUNT(ENROLLMENTID) AS TOTAL_ENROLLMENTS
547 FROM ENROLLMENTS
548 GROUP BY COURSEID
549 HAVING COUNT(ENROLLMENTID) >
550 (
551     SELECT AVG(TOTAL_COUNT)
552     FROM
553     (
554         SELECT COUNT(ENROLLMENTID) AS TOTAL_COUNT
555         FROM ENROLLMENTS
556         GROUP BY COURSEID
557     ) AS COURSE_COUNTS
```

78 %

7 0



Ln: 558, Ch: 2

Results Messages

	COURSEID	TOTAL_ENROLLMENTS
1	503	127
2	505	128
3	509	129
4	508	129
5	504	121
6	501	124
7	506	123

Query executed successfully.

localhost\SQLEXPRESS (17.0 ... RITHIKA\Rithika (65) EdTechLea

SQLProject1....ithika (65))*

```
561      -- Students enrolled in the most expensive course
562
563      SELECT STUDENTID, COURSEID
564      FROM ENROLLMENTS
565      WHERE COURSEID IN
566      (
567          SELECT COURSEID
568          FROM COURSES
569          WHERE COURSEFEE =
570          (
571              SELECT MAX(COURSEFEE)
572              FROM COURSES
573          )
574      )
575
576      -- Highest discount enrollment details
577
578      SELECT *
```

78 %

7 0

Ln: 575,

Results Messages

	STUDENTID	COURSEID
1	1111	508
2	1390	508
3	1087	508
4	1537	508
5	1567	508
6	1594	508
7	1725	508
8	1295	508
9	1755	508
10	1794	508
11	1520	508

Query executed successfully.

localhost\SQLEXPRESS (17.0 ... RITHIKA\Rithika (65) EdTech


```

573     )
574 )
575
576 -- Highest discount enrollment details
577
578 SELECT *
579 FROM ENROLLMENTS
580 WHERE DISCOUNT =
581 (
582     SELECT MAX(DISCOUNT)
583     FROM ENROLLMENTS
584 )
585
586
587 -- Revenue trend for last 6 months using CTE
588
589 WITH REVENUE_CTE AS
590 (

```

78 %

7 0

Results Messages

	ENROLLMENTID	STUDENTID	COURSEID	ENROLLMENTDATE	DISCOUNT	PAYMENTSTATUS
1	2006	1459	509	2025-04-06	30.00	Unpaid
2	2013	1410	510	2025-09-08	30.00	Paid
3	2028	1147	510	2025-04-19	30.00	Unpaid
4	2030	1750	509	2025-07-11	30.00	Unpaid
5	2036	1695	503	2025-12-18	30.00	Paid
6	2045	1312	505	2025-07-08	30.00	Partially Paid
7	2049	1714	502	2025-12-05	30.00	Unpaid
8	2064	1516	506	2025-04-05	30.00	Paid
9	2066	1364	501	2025-01-02	30.00	Paid
10	2067	1382	502	2025-01-22	30.00	Partially Paid
11	2069	1152	506	2025-11-24	30.00	Partially Paid

Query executed successfully.

localhost\SQLEXPRESS (17.0 ... RITHIKA\Rithika (6


```
585
586
587 -- Revenue trend for last 6 months using CTE
588
589 WITH REVENUE_CTE AS
590 (
591     SELECT
592         MONTH(PAYMENTDATE) AS MONTH_NUMBER,
593         SUM(AMOUNTPAID) AS TOTAL_REVENUE
594     FROM PAYMENTS
595     GROUP BY MONTH(PAYMENTDATE)
596 )
597
598 SELECT *
599 FROM REVENUE_CTE
600 ORDER BY MONTH_NUMBER DESC
601
602 -- PART 8 - WINDOW FUNCTIONS
```

78 %



7



0



Results Messages

	MONTH_NUMBER	TOTAL_REVENUE
1	12	3365390.00
2	11	3211283.00
3	10	3301073.00
4	9	3001606.00
5	8	3311610.00
6	7	3047863.00
7	6	3192431.00
8	5	2952099.00
9	4	3855806.00
10	3	4465133.00
11	2	3716072.00

✓ Query executed successfully.



localhost\SQLEXPRESS (17.)


```
601
602      -- PART 8 - WINDOW FUNCTIONS
603      --- RANK() revenue per month
604
605      SELECT
606      MONTH(PAYMENTDATE) AS MONTH_NUMBER,
607      SUM(AMOUNTPAID) AS TOTAL_REVENUE,
608
609      RANK() OVER
610      (
611          ORDER BY SUM(AMOUNTPAID) DESC
612      ) AS REVENUE_RANK
613
614      FROM PAYMENTS
615
616      GROUP BY MONTH(PAYMENTDATE)
617
618      -- DENSE_RANK() - Top 3 courses per month
```

78 %



7



0



Results Messages

	MONTH_NUMBER	TOTAL_REVENUE	REVENUE_RANK
1	3	4465133.00	1
2	1	4337543.00	2
3	4	3855806.00	3
4	2	3716072.00	4
5	12	3365390.00	5
6	8	3311610.00	6
7	10	3301073.00	7
8	11	3211283.00	8
9	6	3192431.00	9
10	7	3047863.00	10
11	9	3001606.00	11

Query executed successfully.



localhost\SOLEXPRESS (17.


```

618 -- DENSE_RANK() - Top 3 courses per month
619
620 SELECT
621     MONTH(P.PAYMENTDATE) AS MONTH_NUMBER,
622     E.COURSEID,
623     SUM(P.AMOUNTPAID) AS TOTAL_REVENUE,
624
625     DENSE_RANK() OVER
626     (
627         PARTITION BY MONTH(P.PAYMENTDATE)
628
629         ORDER BY SUM(P.AMOUNTPAID) DESC
630     ) AS COURSE_RANK
631
632 FROM PAYMENTS P
633
634 INNER JOIN ENROLLMENTS E
635 ON P.ENROLLMENTID = E.ENROLLMENTID
    
```

78 % 7 0

Ln: 639,

Results Messages

	MONTH_NUMBER	COURSEID	TOTAL_REVENUE	COURSE_RANK
1	1	508	758168.00	1
2	1	504	646230.00	2
3	1	510	534651.00	3
4	1	501	447386.00	4
5	1	507	438099.00	5
6	1	509	381298.00	6
7	1	502	342564.00	7
8	1	505	302711.00	8
9	1	503	283288.00	9
10	1	506	203148.00	10
11	2	501	522772.00	1

Query executed successfully.

localhost\SQLEXPRESS (17.0 ... RITHIKA\Rithika (65) EdTechLearnPro 00:00:00

SQLProject1....ithika (65))* ✨ ✕

```

642      -- Compare current month revenue with previous month
643
644      ✓ SELECT
645      MONTH(PAYMENTDATE) AS MONTH_NUMBER,
646
647      SUM(AMOUNTPAID) AS CURRENT_MONTH_REVENUE,
648
649      LAG(SUM(AMOUNTPAID))
650      OVER
651      (
652          ORDER BY MONTH(PAYMENTDATE)
653      ) AS PREVIOUS_MONTH_REVENUE
654
655      FROM PAYMENTS
656
657      GROUP BY MONTH(PAYMENTDATE)
658
659      -- LEAD() to forecast next month trend

```

78 % ▾

✖ 7 ⚠ 0 | ↑ ↓

📊 Results 📄 Messages

	MONTH_NUMBER	CURRENT_MONTH_REVENUE	PREVIOUS_MONTH_REVENUE
1	1	4337543.00	NULL
2	2	3716072.00	4337543.00
3	3	4465133.00	3716072.00
4	4	3855806.00	4465133.00
5	5	2952099.00	3855806.00
6	6	3192431.00	2952099.00
7	7	3047863.00	3192431.00
8	8	3311610.00	3047863.00
9	9	3001606.00	3311610.00
10	10	3301073.00	3001606.00
11	11	3211283.00	3301073.00

✅ Query executed successfully.

🔒 localhost\SQLEXPRESS (17.0 ... | RITHIKA

SQLProject1....ithika (65))*

```

657     GROUP BY MONTH(PAYMENTDATE)
658
659     -- LEAD() to forecast next month trend
660     SELECT
661     MONTH(PAYMENTDATE) AS MONTH_NUMBER,
662
663     SUM(AMOUNTPAID) AS CURRENT_MONTH_REVENUE,
664
665     LEAD(SUM(AMOUNTPAID))
666     OVER
667     (
668         ORDER BY MONTH(PAYMENTDATE)
669     ) AS NEXT_MONTH_REVENUE
670
671     FROM PAYMENTS
672
673     GROUP BY MONTH(PAYMENTDATE)
674

```

78 % 7 0

Results Messages

	MONTH_NUMBER	CURRENT_MONTH_REVENUE	NEXT_MONTH_REVENUE
1	1	4337543.00	3716072.00
2	2	3716072.00	4465133.00
3	3	4465133.00	3855806.00
4	4	3855806.00	2952099.00
5	5	2952099.00	3192431.00
6	6	3192431.00	3047863.00
7	7	3047863.00	3311610.00
8	8	3311610.00	3001606.00
9	9	3001606.00	3301073.00
10	10	3301073.00	3211283.00
11	11	3211283.00	3365390.00

Query executed successfully.

localhost\SQLEXPRESS (17.0 ... RITHIKA\Rithika

SQLProject1....ithika (65))*

```
675 --ROW_NUMBER() - Duplicate detection
676 -- Detect duplicate enrollments
677
678 SELECT
679
680 STUDENTID,
681 COURSEID,
682
683 ROW_NUMBER() OVER
684 (
685     PARTITION BY STUDENTID, COURSEID
686
687     ORDER BY STUDENTID
688 ) AS ROW_NUM
689
690 FROM ENROLLMENTS
691
692
```

78 %



7



0



Ln: 691, Ch: 1 | (155 chars, 13

Results Messages

	STUDENTID	COURSEID	ROW_NUM
1	1001	501	1
2	1001	501	2
3	1001	506	1
4	1001	508	1
5	1002	501	1
6	1002	501	2
7	1003	503	1
8	1007	504	1
9	1007	510	1
10	1008	501	1
11	1008	502	1

✓ Query executed successfully.



localhost\SQLEXPRESS (17.0 ...

RITHIKA\Rithika (65)

EdTechLearnPro

00:00:00 R

SQLProject1....ithika (65))*

```

693 --PART 9 - VIEWS
694 --vw_MonthlyRevenue
695
696 -- Create monthly revenue view
697
698 CREATE VIEW vw_MonthlyRevenue AS
699
700 SELECT
701
702 MONTH(PAYMENTDATE) AS MONTH_NUMBER,
703
704 SUM(AMOUNTPAID) AS TOTAL_REVENUE,
705
706 COUNT(PAYMENTID) AS TOTAL_PAYMENTS
707
708 FROM PAYMENTS
709
710 GROUP BY MONTH(PAYMENTDATE)
711
712 SELECT *
713 FROM vw_MonthlyRevenue
714
715
716 -- Create trainer performance view

```

59 % 99+ 0

Results Messages

	MONTH_NUMBER	TOTAL_REVENUE	TOTAL_PAYMENTS
1	9	3001606.00	90
2	3	4465133.00	124
3	12	3365390.00	93
4	6	3192431.00	91
5	7	3047863.00	93
6	1	4337543.00	124
7	10	3301073.00	93
8	4	3855806.00	104
9	5	2952099.00	93
10	2	3716072.00	113
11	11	3211283.00	90

Query executed successfully.

localhost\SQLEXPRESS (17.0 ...

SQLProject1....ithika (65))*

714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742

-- Create trainer performance view

CREATE VIEW vw_TrainerPerformance AS

SELECT

T.TRAINERNAME,

T.EXPERTISE,

COUNT(TCM.COURSEID) AS TOTAL_ASSIGNED_COURSES,

ROUND(AVG(T.SATISFACTIONSORE),2) AS AVG_SATISFACTION

FROM TRAINERS T

LEFT JOIN TRAINER_COURSE_MAPPING TCM
ON T.TRAINERID = TCM.TRAINERID

GROUP BY
T.TRAINERNAME,
T.EXPERTISE

SELECT *
FROM vw_TrainerPerformance

48 %

99+0

Ln: 740, Ch: 27 (36 chars, 2 lines) SPC CRLF UTF-8

ResultsMessages

	TRAINERNAME	EXPERTISE	TOTAL_ASSIGNED_COURSES	AVG_SATISFACTION
1	Trainer_3	AI	1	4.400000
2	Trainer_5	AI	1	3.870000
3	Trainer_10	Cloud	1	3.990000
4	Trainer_1	Power BI	5	4.190000
5	Trainer_4	Power BI	1	4.410000
6	Trainer_6	Power BI	3	4.340000
7	Trainer_9	Power BI	4	4.480000
8	Trainer_2	Python	4	3.500000
9	Trainer_8	Python	4	3.740000
10	Trainer_7	SQL	0	3.840000

Query executed successfully.

localhost\SQLEXPRESS (17.0 ... RITHIKA\Rithika (65) EdTechLearnPro 00:00:00 Row: 1, Col: 1 10 rows

SQLProject1....ithika (65)*

```
744 -- Create:
745 -- Index on Students.Email
746 -- Index on Courses.CourseName
747 -- Index on Enrollments.EnrollmentDate
748 -- Composite index (StudentID, CourseID)
749
750
751 -- Index on Students.Email
752
753 CREATE INDEX IDX_STUDENTS_EMAIL
754 ON STUDENTS(EMAIL)
755 --This index helps when searching students using email.
756
757 --Example
758 SELECT *
759 FROM STUDENTS
760 WHERE EMAIL = 'student1@example.com'
761 --Instead of scanning the entire STUDENTS table, SQL directly locates the required email.
762
763
764
765 -- Index on Courses.CourseName
766
767 CREATE INDEX IDX_COURSES_COURSENAME
768 ON COURSES(COURSENAME)
769 --Useful when searching courses by name.
770
771 --Example
772 SELECT *
```

48 % 99+ 0

Results Messages

	STUDENTID	FULLNAME	EMAIL	PHONE	CITY	STATE	REGISTRATIONDATE	ISACTIVE
1	1001	Student_1	student1@example.com	9634895718	Delhi	Delhi	2024-01-01 00:00:00.000	1

SQLProject1....ithika (65))*

```

762
763
764
765 -- Index on Courses.CourseName
766
767 CREATE INDEX IDX_COURSES_COURSENAME
768 ON COURSES(COURSENAME)
769 --Useful when searching courses by name.
770
771 --Example
772 SELECT *
773 FROM COURSES
774 WHERE COURSENAME = 'Python'
775 --SQL can find the course much faster.
776
777
778 ---- Index on Enrollments.EnrollmentDate
779
780 CREATE INDEX IDX_ENROLLMENTS_DATE
781 ON ENROLLMENTS(ENROLLMENTDATE)
782 --Useful for date filtering and reports.
783
784 --Example
785 SELECT *
786 FROM ENROLLMENTS
787 WHERE ENROLLMENTDATE
788 BETWEEN '2025-01-01' AND '2025-03-31'
789 --SQL quickly locates rows within the date range.
790

```

48 % 99+ 0

Results Messages

COURSEID	COURSENAME	CATEGORY	COURSEFEE	DIFFICULTYLEVEL
----------	------------	----------	-----------	-----------------


```
762
763
764
765 -- Index on Courses.CourseName
766
767 CREATE INDEX IDX_COURSES_COURSENAME
768 ON COURSES(COURSENAME)
769 --Useful when searching courses by name.
770
771 --Example
772 SELECT *
773 FROM COURSES
774 WHERE COURSENAME = 'Python'
775
776 SELECT *
777 FROM COURSES
778 WHERE COURSENAME LIKE '%Python%'
779 --SQL can find the course much faster.
780
781
782 ---- Index on Enrollments.EnrollmentDate
783
784 CREATE INDEX IDX_ENROLLMENTS_DATE
785 ON ENROLLMENTS(ENROLLMENTDATE)
786 --Useful for date filtering and reports.
787
788 --Example
789 SELECT *
790 FROM ENROLLMENTS
```

48 %



7



0



Results Messages

	COURSEID	COURSENAME	CATEGORY	COURSEFEE	DIFFICULTYLEVEL
1	505	Python Full Stack	Programming	55000	Intermediate


```
762
763
764
765 -- Index on Courses.CourseName
766
767 CREATE INDEX IDX_COURSES_COURSENAME
768 ON COURSES(COURSENAME)
769 --Useful when searching courses by name.
770
771 --Example
772
773 SELECT *
774 FROM COURSES
775 WHERE COURSENAME LIKE '%Python%'
776 --SQL can find the course much faster.
777
778
779 ---- Index on Enrollments.EnrollmentDate
780
781 CREATE INDEX IDX_ENROLLMENTS_DATE
782 ON ENROLLMENTS(ENROLLMENTDATE)
783 --Useful for date filtering and reports.
784
785 --Example
786 SELECT *
787 FROM ENROLLMENTS
788 WHERE ENROLLMENTDATE
789 BETWEEN '2025-01-01' AND '2025-03-31'
790 --SQL quickly locates rows within the date range.
```

48 % 7 0

Results Messages

	COURSEID	COURSENAME	CATEGORY	COURSEFEE	DIFFICULTYLEVEL
1	505	Python Full Stack	Programming	55000	Intermediate

SQLProject1....ithika (65))*

```

774 FROM COURSES
775 WHERE COURSENAME LIKE '%Python%'
776 --SQL can find the course much faster.
777
778
779 ---- Index on Enrollments.EnrollmentDate
780
781 CREATE INDEX IDX_ENROLLMENTS_DATE
782 ON ENROLLMENTS(ENROLLMENTDATE)
783 --Useful for date filtering and reports.
784
785 --Example
786 SELECT *
787 FROM ENROLLMENTS
788 WHERE ENROLLMENTDATE
789 BETWEEN '2025-01-01' AND '2025-03-31'
790 --SQL quickly locates rows within the date range.
791
792 -- Composite Index
793
794 CREATE INDEX IDX_STUDENT_COURSE
795 ON ENROLLMENTS(STUDENTID, COURSEID)
796 --A composite index improves query performance by allowing SQL Server to quickly locate records when both StudentID and CourseID are used together in searches, filters, or joins.
797
798 --Example
799 SELECT *
800 FROM ENROLLMENTS
801 WHERE STUDENTID = 1001
802 AND COURSEID = 501

```

48 %

7 0

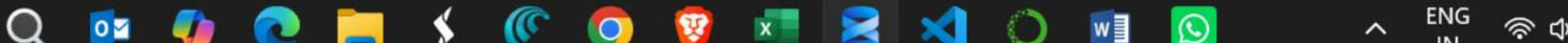
Ln: 786, Ch: 1 (87 chars, 4

Results Messages

	ENROLLMENTID	STUDENTID	COURSEID	ENROLLMENTDATE	DISCOUNT	PAYMENTSTATUS
1	2001	1318	507	2025-03-22	25.00	Paid
2	2002	1111	508	2025-03-29	5.00	Partially Paid
3	2008	1208	509	2025-01-01	25.00	Partially Paid
4	2012	1022	502	2025-03-22	25.00	Partially Paid
5	2015	1515	505	2025-03-14	0.00	Partially Paid
6	2017	1809	502	2025-03-11	5.00	Partially Paid
7	2022	1087	508	2025-01-03	25.00	Partially Paid
8	2024	1537	508	2025-03-04	25.00	Unpaid
9	2025	1301	507	2025-02-18	10.00	Partially Paid
10	2026	1243	505	2025-03-17	25.00	Partially Paid
11	2031	1356	505	2025-03-06	25.00	Paid

Query executed successfully.

localhost\SQLEXPRESS (17.0 ... RITHIKA\Rithika (65) EdTechLearnPro 00:00:01



SQLProject1....ithika (65))*

```
789 BETWEEN '2025-01-01' AND '2025-03-31'
790 --SQL quickly locates rows within the date range.
791
792 -- Composite Index
793
794 CREATE INDEX IDX_STUDENT_COURSE
795 ON ENROLLMENTS(STUDENTID, COURSEID)
796 --A composite index improves query performance by allowing SQL Server to quickly locate records when both StudentID and CourseID are used together in searches, filters, or joins.
797
798 --Example
799 SELECT *
800 FROM ENROLLMENTS
801 WHERE STUDENTID = 1001
802 AND COURSEID = 501
803
804
805 --PART 11 - STORED PROCEDURES
806 --Create procedures for:
807 --1. Fetching monthly revenue (IN parameter)
808
809 CREATE PROCEDURE sp_MonthlyRevenue
810 @Month INT
811 AS
812 BEGIN
813
814     SELECT
815         MONTH(PaymentDate) AS MonthNo,
816         SUM(AmountPaid) AS TotalRevenue
817     FROM Payments
```

48 % 7 0 Ln: 799, Ch: 1 (70 chars, 4 lines)

Results Messages

	ENROLLMENTID	STUDENTID	COURSEID	ENROLLMENTDATE	DISCOUNT	PAYMENTSTATUS
1	9998	1001	501	2026-06-15	10.00	Unpaid
2	9999	1001	501	2026-06-15	20.00	Paid

SQLProject1....ithika (65))*

```
802 AND COURSEID = 501
803
804
805 --PART 11 - STORED PROCEDURES
806 --Create procedures for:
807 --1. Fetching monthly revenue (IN parameter)
808
809 CREATE PROCEDURE sp_MonthlyRevenue
810     @Month INT
811 AS
812 BEGIN
813
814     SELECT
815         MONTH(PaymentDate) AS MonthNo,
816         SUM(AmountPaid) AS TotalRevenue
817     FROM Payments
818     WHERE MONTH(PaymentDate) = @Month
819     GROUP BY MONTH(PaymentDate)
820
821 END
822
823
824 EXEC sp_MonthlyRevenue 5
825
826
827
828 --2. Adding a new student with validation (INOUT)
829
830 CREATE PROCEDURE sp_AddStudent
```

48 %



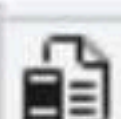
99+



0



Results



Messages

	MonthNo	TotalRevenue
1	5	2952099.00

SQLProject1...ithika (65))*

--2. Adding a new student with validation (INOUT)

```
CREATE PROCEDURE sp_AddStudent
(
    @StudentID INT,
    @FullName VARCHAR(100),
    @Email VARCHAR(100),
    @Phone VARCHAR(20),
    @City VARCHAR(50),
    @State VARCHAR(50),
    @Message VARCHAR(100) OUTPUT
)
AS
BEGIN
    IF EXISTS
    (
        SELECT *
        FROM Students
        WHERE Email = @Email
    )
    BEGIN
        SET @Message='Email already exists'
    END
    ELSE
    BEGIN
        INSERT INTO Students
        (
```

40 %



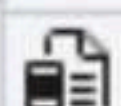
99+



0



Results



Messages

(No column name)

1

Email already exists



SQLProject1....ithika (65))*

```
870      @StudentID,  
871      @FullName,  
872      @Email,  
873      @Phone,  
874      @City,  
875      @State  
876    )  
877  
878    SET @Message='Student Added Successfully'  
879  
880  END  
881  
882 END  
883  
884  
885 DECLARE @Result VARCHAR(100)  
886  
887 EXEC sp_AddStudent  
888 10000,  
889 'Rithika',  
890 'rithika@gmail.com',  
891 '9876543210',  
892 'Hyderabad',  
893 'Telangana',  
894 @Result OUTPUT  
895  
896 SELECT @Result  
897  
898 --3.Getting trainer performance above a threshold (OUT)  
899  
900  
901 CREATE PROCEDURE sp_TrainerPerformance  
902 (  
903   @Threshold DECIMAL(4,2),  
904   @TotalTrainers INT OUTPUT
```

40 % 99+ 0 Ln: 8

Results Messages

(No column name)	
1	Email already exists


```
898
899 --3.Getting trainer performance above a threshold (OUT)
900
901 CREATE PROCEDURE sp_TrainerPerformance
902 (
903     @Threshold DECIMAL(4,2),
904     @TotalTrainers INT OUTPUT
905 )
906 AS
907 BEGIN
908
909     SELECT
910         @TotalTrainers = COUNT(*)
911     FROM Trainers
912     WHERE SatisfactionScore > @Threshold
913
914 END
915
916 DECLARE @Count INT
917
918 EXEC sp_TrainerPerformance
919 4.0,
920 @Count OUTPUT
921
922 SELECT @Count AS TrainersAboveThreshold
923
924
```

	TrainersAboveThreshold
1	5


```
898
899 --3.Getting trainer performance above a threshold (OUT)
900
901 CREATE PROCEDURE sp_TrainerPerformance
902 (
903     @Threshold DECIMAL(4,2),
904     @TotalTrainers INT OUTPUT
905 )
906 AS
907 BEGIN
908
909     SELECT
910         @TotalTrainers = COUNT(*)
911     FROM Trainers
912     WHERE SatisfactionScore > @Threshold
913
914 END
915
916 DECLARE @Count INT
917
918 EXEC sp_TrainerPerformance
919 4.0,
920 @Count OUTPUT
921
922 SELECT @Count AS TrainersAboveThreshold
923
924
```

Results Messages

	TrainersAboveThreshold
1	5


```
925 --PART 12 - USER DEFINED FUNCTIONS
926 --Create:
927 --1. Scalar Function + fn_NetFeeAfterDiscount
928
929 CREATE FUNCTION fn_NetFeeAfterDiscount
930 (
931     @CourseFee DECIMAL(10,2),
932     @Discount DECIMAL(10,2)
933 )
934
935 RETURNS DECIMAL(10,2)
936
937 AS
938 BEGIN
939
940     DECLARE @NetFee DECIMAL(10,2)
941
942     SET @NetFee =
943     @CourseFee - (@CourseFee * @Discount / 100)
944
945     RETURN @NetFee
946
947 END
948
949
950 SELECT dbo.fn_NetFeeAfterDiscount
951 (
952     50000,
953     10
954 )
955 AS NET_FEE
```

53 %

✖ 99+

⚠ 0

↑ ↓

Results Messages

	NET_FEE
1	45000.00

SQLProject1....ithika (58))*

```

955 AS NET_FEE
956
957
958 --2. Table Valued Function + return enrollments by course
959 CREATE FUNCTION fn_EnrollmentsByCourse
960 (
961     @CourseID INT
962 )
963
964 RETURNS TABLE
965
966 AS
967
968 RETURN
969 (
970
971     SELECT
972     ENROLLMENTID,
973     STUDENTID,
974     COURSEID,
975     ENROLLMENTDATE
976
977     FROM ENROLLMENTS
978
979     WHERE COURSEID = @CourseID
980 )
981
982
983 SELECT *
984 FROM dbo.fn_EnrollmentsByCourse(501)
985

```

53 %



99+



0



Ln:

Results Messages

	ENROLLMENTID	STUDENTID	COURSEID	ENROLLMENTDATE
1	2018	1700	501	2025-09-19
2	2041	1723	501	2025-11-10
3	2048	1643	501	2025-11-11
4	2057	1817	501	2025-11-16
5	2058	1862	501	2025-05-27
6	2065	1732	501	2025-05-31
7	2066	1364	501	2025-01-02

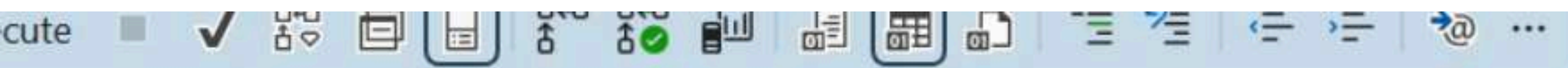
Query executed successfully.



localhost\SQLEXPRESS (17.0 ...

RITHIKA\Rithika (58)

EdT



SQLProject1...ithika (58))*

- RIT

```
987 --PART 13 - EXCEPTION HANDLING
988 --Use TRY_CATCH inside stored procedures:
989 --Log errors into ErrorLog table
990 --Insert user friendly error messages
991
992 --Step 1: Create ErrorLog Table
993 CREATE TABLE ERRORLOG
994 (
995     ERRORID INT IDENTITY(1,1) PRIMARY KEY,
996     ERRORMESSAGE VARCHAR(500),
997     ERRORDATE DATETIME DEFAULT GETDATE()
998 )
999
1000 --Whenever an error occurs, the error message gets stored here.
1001
1002 --Step 2: Stored Procedure with TRY...CATCH
1003 CREATE PROCEDURE SP_MONTHLYREVENUE_ERRORHANDLING
1004
1005     @MONTH INT
1006
1007 AS
1008 BEGIN
1009
1010     BEGIN TRY
1011
1012         SELECT
1013             SUM(AMOUNTPAID) AS TOTAL_REVENUE
1014
1015         FROM PAYMENTS
1016
1017         WHERE MONTH(PAYMENTDATE) = @MONTH
1018
```

53 % 99+ 0

Ln: 1043, Ch: 1 (25 chars, 2 l

Results Messages

ERRORID	ERRORMESSAGE	ERRORDATE
---------	--------------	-----------

Query executed successfully.

localhost\SQLEXPRESS (17.0 ... RITHIKA\Rithika (58) EdTechLearnPro 00:00:00

SQLProject1....ithika (58))*

```

901
902
903 SELECT *
904 FROM dbo.fn_EnrollmentssByCourse(501)
905
906
907 --PART 12 - EXCEPTION HANDLING
908 --Use TRY...CATCH inside stored procedures:
909 --Log errors into ErrorLog table
910 --Insert user friendly error messages
911
912 --Step 1: Create ErrorLog Table
913 CREATE TABLE ERRORLOG
914 (
915     ERRORID INT IDENTITY(1,1) PRIMARY KEY,
916     ERRORMESSAGE VARCHAR(500),
917     ERRORDATE DATETIME DEFAULT GETDATE()
918 )
919
920 --Whenever an error occurs, the error message gets stored here.
921
922 --Step 2: Stored Procedure with TRY...CATCH
923 CREATE PROCEDURE SP_MONTHLYREVENUE_ERRORHANDLING
924     @MONTH INT
925 AS
926 BEGIN
927     BEGIN TRY
928         SELECT
929             SUM(AMOUNTPAID) AS TOTAL_REVENUE
930         FROM PAYMENTS
931         WHERE MONTH(PAYMENTDATE) = @MONTH
932     END TRY
933     BEGIN CATCH
934         INSERT INTO ERRORLOG
935         (
936             ERRORMESSAGE
937         )
938         VALUES
939         (
940             ERROR_MESSAGE()
941         )
942         PRINT 'Revenue could not be calculated.'
943     END CATCH
944 END
945
946 EXEC SP_MONTHLYREVENUE_ERRORHANDLING 1
947
948 --checking errorlog
949
950 SELECT *
951 FROM ERRORLOG

```

27 %



99+



0



Results Messages

	TOTAL_REVENUE
1	4337543.00



SQLProject1....ithika (58))*

```
985
986
987 --PART 13 - EXCEPTION HANDLING
988 --Use TRY...CATCH inside stored procedures:
989 --Log errors into ErrorLog table
990 --Insert user friendly error messages
991
992 --Step 1: Create ErrorLog Table
993 CREATE TABLE ERRORLOG
994 (
995     ERRORID INT IDENTITY(1,1) PRIMARY KEY,
996     ERRORMESSAGE VARCHAR(500),
997     ERRORDATE DATETIME DEFAULT GETDATE()
998 )
999
1000 --Whenever an error occurs, the error message gets stored here.
1001
1002 --Step 2: Stored Procedure with TRY...CATCH
1003 CREATE PROCEDURE SP_MONTHLYREVENUE_ERRORHANDLING
1004
1005     @MONTH INT
1006
1007 AS
1008 BEGIN
1009
1010     BEGIN TRY
```

64 %

99+ 0

Ln: 1046, Ch: 1 (23 chars, 2 lines) SPC

Results Messages

	ERRORID	ERRORMESSAGE	ERRORDATE
1	1	Invalid month entered.	2026-06-17 21:29:53.783

SQLProject1....ithika (58))*

1047 FROM ERRORLOG

1048

1049

1050 --PART 14 - TRIGGERS

1051 --Create:

1052 --1. BEFORE INSERT → prevent discount > 50%

1053

1054 CREATE TABLE AUDITLOG

1055 (

1056 AUDITID INT IDENTITY(1,1) PRIMARY KEY,

1057 ENROLLMENTID INT,

1058 OLDDISCOUNT DECIMAL(10,2),

1059 NEWDISCOUNT DECIMAL(10,2),

1060 CHANGEDATE DATETIME DEFAULT GETDATE()

1061)

1062

1063 CREATE TRIGGER TRG_PREVENT_HIGH_DISCOUNT

1064 ON ENROLLMENTS

1065 AFTER INSERT

1066 AS

1067 BEGIN

1068

1069 IF EXISTS

1070 (

1071 SELECT *

1072 FROM INSERTED

1073 WHERE DISCOUNT > 50

1074)

1075

1076 BEGIN

1077

1078 RAISERROR

1079 (

1080 'Discount cannot exceed 50 percent',

1081 16,

48 %

99+

0

↑

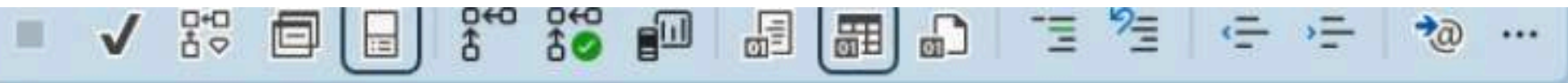
↓

Ln: 1080, Ch: 24

Results

Messages

	ENROLLMENTID	STUDENTID	COURSEID	ENROLLMENTDATE	DISCOUNT	PAYMENTSTATUS
1	9992	1001	501	2026-06-17	20.00	Paid



SQLProject1....ithika (58)*

```
1116 -- Trigger prevents insertion when discount exceeds 50%.
1117 -- If discount > 50, transaction is rolled back.
1118
1119 CREATE TRIGGER TRG_UNPAID_PAYMENT
1120
1121 ON ENROLLMENTS
1122
1123 AFTER INSERT
1124
1125 AS
1126 BEGIN
1127
1128     INSERT INTO PAYMENTS
1129     (
1130         PAYMENTID,
1131         ENROLLMENTID,
1132         AMOUNTPAID,
1133         PAYMENTDATE
1134     )
1135
1136     SELECT
1137         (SELECT ISNULL(MAX(PAYMENTID),3000) + 1 FROM PAYMENTS),
1138         ENROLLMENTID,
1139         0,
1140         GETDATE()
1141
1142     FROM INSERTED
1143
1144     WHERE PAYMENTSTATUS = 'Unpaid'
1145
1146 END
1147
1148 INSERT INTO ENROLLMENTS
1149 (
1150
```

48 % 99+ 0 Ln: 1168, Ch: 1 (52 chars, 4 lines) SPC CRLF UTF

Results Messages

	PAYMENTID	ENROLLMENTID	AMOUNTPAID	PAYMENTDATE
1	4202	9996	0.00	2026-06-17 21:38:57.147

SQLProject1....ithika (58))*

1170 FROM PAYMENTID
1171 WHERE ENROLLMENTID = 9996
1172
1173
1174 --3. AFTER UPDATE + log old + new values into AuditLog table
1175
1176 CREATE TRIGGER TRG_AUDIT_UPDATE
1177
1178 ON ENROLLMENTS
1179
1180 AFTER UPDATE
1181
1182 AS
1183 BEGIN
1184 INSERT INTO AUDITLOG
1185 (
1186 ENROLLMENTID,
1187 OLDDISCOUNT,
1188 NEWDISCOUNT,
1189 CHANGEDATE
1190)
1191 SELECT
1192 D.ENROLLMENTID,
1193 D.DISCOUNT,
1194 I.DISCOUNT,
1195 GETDATE()
1196
1197 FROM DELETED D
1198
1199 INNER JOIN INSERTED I
1200 ON D.ENROLLMENTID = I.ENROLLMENTID
1201
1202 END
1203
1204
1205 UPDATE ENROLLMENTS
1206 SET DISCOUNT = 25
1207 WHERE ENROLLMENTID = 2001
1208
1209 SELECT *
1210 FROM AUDITLOG
1211

40 %

✖ 99+ ⚠ 0 ⬆ ⬇

Ln: 1209, Ch: 1 (23 chars, 2 lines) SPC CRLF UTF-8

Results Messages

	AUDITID	ENROLLMENTID	OLDDISCOUNT	NEWDISCOUNT	CHANGEDATE
1	1	2001	25.00	25.00	2026-06-17 23:03:06.390

✔ Query executed successfully.

🔒 localhost\SQLEXPRESS (17.0 ... RITHIKA\Rithika (58) EdTechLearnPro 00:00:00 Row: 1, Col: 1 1 rows


```
1212
1213 --PART 15 - FINAL ANALYTICAL TASKS
1214 --Produce SQL results for:
1215 --CAC (Cost per Acquisition)- ROI per marketing channel
1216
1217 SELECT
1218     M.CHANNEL,
1219     SUM(M.SPEND) AS TOTAL_SPEND,
1220     COUNT(E.ENROLLMENTID) AS TOTAL_ENROLLMENTS,
1221     SUM(M.SPEND) / COUNT(E.ENROLLMENTID) AS CAC
1222
1223 FROM MARKETING M
1224
1225 LEFT JOIN ENROLLMENTS E
1226 ON FORMAT(E.ENROLLMENTDATE, 'yyyy-MM') = M.MONTHYEAR
1227
1228 GROUP BY M.CHANNEL
1229
1230 HAVING COUNT(E.ENROLLMENTID) > 0
1231
1232 ORDER BY CAC
1233
1234 --- 12 month enrollment heatmap (Month x City)
1235
1236 SELECT
1237     FORMAT(E.ENROLLMENTDATE, 'yyyy-MM') AS MONTHYEAR,
1238     S.CITY,
1239     COUNT(E.ENROLLMENTID) AS TOTAL_ENROLLMENTS
1240
1241 FROM ENROLLMENTS E
1242
1243 INNER JOIN STUDENTS S
```

Results Messages

	CHANNEL	TOTAL_SPEND	TOTAL_ENROLLMENTS	CAC
1	Email	168253348.00	1200	140211.123333
2	Google Ads	170388969.00	1200	141990.807500
3	Meta Ads	192916715.00	1200	160763.929166
4	YouTube	201116256.00	1200	167596.880000
5	Influencers	226625194.00	1200	188854.328333
6	LinkedIn	230608051.00	1200	192173.375833


```
1233  
1234 --- 12 month enrollment heatmap (Month x City)  
1235  
1236 SELECT  
1237     FORMAT(E.ENROLLMENTDATE, 'yyyy-MM') AS MONTHYEAR,  
1238     S.CITY,  
1239     COUNT(E.ENROLLMENTID) AS TOTAL_ENROLLMENTS  
1240  
1241 FROM ENROLLMENTS E  
1242  
1243 INNER JOIN STUDENTS S  
1244 ON E.STUDENTID = S.STUDENTID  
1245  
1246 GROUP BY  
1247     FORMAT(E.ENROLLMENTDATE, 'yyyy-MM'),  
1248     S.CITY  
1249  
1250 ORDER BY  
1251     MONTHYEAR,  
1252     TOTAL_ENROLLMENTS DESC  
1253
```

78 %

99+ 0



Ln: 1252, Ch: 23 (303)

Results Messages

	MONTHYEAR	CITY	TOTAL_ENROLLMENTS
1	2025-01	Chennai	13
2	2025-01	Kolkata	12
3	2025-01	Goa	11
4	2025-01	Delhi	9
5	2025-01	Ahmedabad	8
6	2025-01	Bangalore	8
7	2025-01	Hyderabad	6
8	2025-01	Jaipur	6
9	2025-01	Mumbai	6

Query executed successfully.

localhost\SQLEXPRESS (17.0 ... RITHIKA\Rithika (58) EdTechLearnPro 0

SQLProject1...ithika (58)*

```
1251 MONTHYEAR,  
1252 TOTAL_ENROLLMENTS DESC  
1253  
1254 --- Top course & worst course by revenue |  
1255 SELECT  
1256     TOP 1  
1257     E.COURSEID,  
1258     SUM(P.AMOUNTPAID) AS TOTAL_REVENUE  
1259  
1260 FROM ENROLLMENTS E  
1261  
1262 INNER JOIN PAYMENTS P  
1263 ON E.ENROLLMENTID = P.ENROLLMENTID  
1264  
1265 GROUP BY E.COURSEID  
1266  
1267 ORDER BY TOTAL_REVENUE DESC  
1268  
1269  
1270 --Lowest revenue Course  
1271 SELECT
```

78 % 99+ 0

Results Messages

	COURSEID	TOTAL_REVENUE
1	501	4747754.00

SQLProject1....ithika (58))*

```
1269
1270 --Lowest revenue Course
1271 SELECT
1272     TOP 1
1273     E.COURSEID,
1274     SUM(P.AMOUNTPAID) AS TOTAL_REVENUE
1275
1276 FROM ENROLLMENTS E
1277
1278 INNER JOIN PAYMENTS P
1279 ON E.ENROLLMENTID = P.ENROLLMENTID
1280
1281 GROUP BY E.COURSEID
1282
1283 ORDER BY TOTAL_REVENUE ASC
1284
1285 --- Trainer impact on course satisfaction
```

94 % 99+ 0

Results Messages

	COURSEID	TOTAL_REVENUE
1	510	3476370.00


```
1283 ORDER BY TOTAL_REVENUE ASC
1284
1285 --- Trainer impact on course satisfaction
1286
1287 SELECT
1288     T.TRAINERNAME,
1289     T.EXPERTISE,
1290     CAST(AVG(T.SATISFACTIONSORE) AS DECIMAL(10,2))
1291     AS AVG_SATISFACTION
1292
1293 FROM TRAINERS T
1294
1295 GROUP BY
1296     T.TRAINERNAME,
1297     T.EXPERTISE
1298
1299 ORDER BY AVG_SATISFACTION DESC
1300
```

	TRAINERNAME	EXPERTISE	AVG_SATISFACTION
1	Trainer_9	Power BI	4.48
2	Trainer_4	Power BI	4.41
3	Trainer_3	AI	4.40
4	Trainer_6	Power BI	4.34
5	Trainer_1	Power BI	4.19
6	Trainer_10	Cloud	3.99
7	Trainer_5	AI	3.87
8	Trainer_7	SQL	3.84
9	Trainer_8	Python	3.74


```
1301 --- Month over month business growth
1302
1303 SELECT
1304     MONTH(PAYMENTDATE) AS MONTHNO,
1305
1306     SUM(AMOUNTPAID) AS REVENUE,
1307
1308     LAG(SUM(AMOUNTPAID))
1309     OVER(ORDER BY MONTH(PAYMENTDATE))
1310     AS PREVIOUS_MONTH_REVENUE
1311
1312 FROM PAYMENTS
1313
1314 GROUP BY MONTH(PAYMENTDATE)
1315
1316 ORDER BY MONTHNO
1317
1318
```

	MONTHNO	REVENUE	PREVIOUS_MONTH_REVENUE
1	1	4337543.00	NULL
2	2	3716072.00	4337543.00
3	3	4465133.00	3716072.00
4	4	3855806.00	4465133.00
5	5	2952099.00	3855806.00
6	6	3192431.00	2952099.00
7	7	3047863.00	3192431.00
8	8	3311610.00	3047863.00
9	9	2001606.00	3311610.00


```
1301 --- Month over month business growth
1302
1303 SELECT
1304     MONTH(PAYMENTDATE) AS MONTHNO,
1305
1306     SUM(AMOUNTPAID) AS REVENUE,
1307
1308     LAG(SUM(AMOUNTPAID))
1309     OVER(ORDER BY MONTH(PAYMENTDATE))
1310     AS PREVIOUS_MONTH_REVENUE
1311
1312 FROM PAYMENTS
1313
1314 GROUP BY MONTH(PAYMENTDATE)
1315
1316 ORDER BY MONTHNO
1317
1318
```

	MONTHNO	REVENUE	PREVIOUS_MONTH_REVENUE
1	1	4337543.00	NULL
2	2	3716072.00	4337543.00
3	3	4465133.00	3716072.00
4	4	3855806.00	4465133.00
5	5	2952099.00	3855806.00
6	6	3192431.00	2952099.00
7	7	3047863.00	3192431.00
8	8	3311610.00	3047863.00
9	9	2001606.00	3311610.00